Backlink Admissibility in Cyclic Proofs for the Calculus of Inductive Constructions

Gavin Mendel-Gleason

Scidonia Limited
gavin@scidonia.ai

Abstract. Cyclic proofs close goals by backlinking to previously-seen configurations, discovering induction principles post-hoc rather than prescribing them upfront. We prove that backlinks are *admissible* in the Calculus of Inductive Constructions (CIC): every valid cyclic proof can be read back to a standard CIC term that uses only the native fixpoint binder () for recursion and is observationally indistinguishable from the root goal. The proof is a direct corollary of a contextual-equivalence theorem mechanised in Rocq with 0 axioms.

1 Introduction

Cyclic proof theory [1,2] extends standard proof systems with backlink rules that allow a premise to reference a previous goal, forming a directed cycle. The cycle is sound if it satisfies a global *trace condition*: every cycle must contain a *progress event*—a case-split on a structurally smaller term. Brotherston and Simpson prove that for first-order logic with inductive definitions, backlinks are *admissible*: every cyclic proof corresponds to a standard proof with an explicit induction rule.

This paper extends their result to the Calculus of Inductive Constructions (CIC) [3,5], the type theory underlying Rocq. We prove that every rooted, ranked cyclic proof can be residualised to a standard CIC term using only for recursion, and that the residual is observationally equivalent to the root goal.

2 Term calculus and reduction

Terms are given by the grammar:

$$t, u, A, B ::= x \mid i \mid \Pi x{:}A.\, B \mid \lambda x{:}A.\, t \mid t\ u \mid\ \ t \mid I\boldsymbol{t} \mid Ic\boldsymbol{t} \mid It\kappa\boldsymbol{b}$$

where x ranges over de Bruijn indices, i over universe levels, I identifies an inductive family, c a constructor index, κ a dependent motive (a binder over the scrutinee), and $\boldsymbol{t}, \boldsymbol{b}$ are lists of terms.

We write $\to^*$ for the reflexive-transitive closure of $\to$, and $t \Downarrow v$ ("t terminates to v") for $t \to^* v$ with v a value (an abstraction, a constructor, or a neutral term). Multi-argument application $\mathsf{apps}\ t\ [u_1, \ldots, u_n]$ is defined inductively: $\mathsf{apps}\ t\ [] = t$ and $\mathsf{apps}\ t\ (u :: \boldsymbol{u}) = \mathsf{apps}\ (t\ u)\ \boldsymbol{u}$.

Operational semantics. We use a call-by-name (CBN) reduction relation $t \to t'$ defined by:

- β**-reduction:** $(\lambda x{:}A.\, t)\ u \to t[u/x]$.
- **Fixpoint unfolding:** $t \to t[\,t/0]$. (The binder 0 in t refers to the recursive self-reference.)
- ι**-reduction (case-of-constructor):** $I(Ic\boldsymbol{a})\kappa\boldsymbol{b} \to \mathsf{apps}\ (\boldsymbol{b}[c])\ \boldsymbol{a}$ where $\boldsymbol{b}[c]$ selects the c-th branch.
- **Congruence:** Reduction applies under applications and case scrutinees: if $t \to t'$ then $t\ u \to t'\ u$ and $It\kappa\boldsymbol{b} \to It'\kappa\boldsymbol{b}$.

A term t *terminates to* a value v, written $t \Downarrow v$, if $t \to^* v$ and v is in weak-head normal form (an abstraction, a constructor, or a neutral term).

3 Cyclic sequent calculus

Configurations are typing judgements $\Gamma \vdash t : A$. A *configuration graph* b is a finite map from vertices $v \in \{0, \ldots, n-1\}$ to configurations $\ell(b, v)$ and successor lists $\sigma(b, v)$. A graph is a *rooted pre-proof* with root v_0 if every vertex satisfies a local sequent rule drawn from the following:

Asynchronous rules (deterministic). Applied eagerly. Each reduces the goal to a single premise by one computation step:

$$\frac{\Gamma \vdash t[u/x] : A}{\Gamma \vdash (\lambda x{:}A.\, t)\ u : A}\ \beta \qquad \frac{\Gamma \vdash t[\,t/0] : A}{\Gamma \vdash\ t : A}\ \textsc{fix}$$

$$\frac{\Gamma \vdash \mathsf{apps}\ (\boldsymbol{b}[c])\ \boldsymbol{a} : A}{\Gamma \vdash I(Ic\boldsymbol{a})\kappa\boldsymbol{b} : A}\ \iota$$

Synchronous rule (choice point). A case-split on a neutral scrutinee x creates one premise per constructor c of I, each in an extended context with fresh constructor arguments $\boldsymbol{y}_c$:

$$\frac{\Gamma, \boldsymbol{y}_0 \vdash t_0 : A \quad \cdots \quad \Gamma, \boldsymbol{y}_n \vdash t_n : A}{\Gamma \vdash Ix\kappa\boldsymbol{b} : A}\ \textsc{split}$$

Cyclic backlink (closure). $\dfrac{\Delta \vdash \sigma :_s \Gamma}{\Delta \vdash t : A}\ \textsc{back}$ where t is an instance of a previously-seen configuration $\Gamma \vdash t_0 : A$ under the explicit substitution σ.

Global progress condition. A rooted pre-proof is a *rooted cyclic proof* if every directed cycle contains at least one SPLIT edge. This is witnessed by a budget-trace ranking: vertices are lifted to pairs (v, k) where $k \in [0, |V|]$. SPLIT edges consume budget $(k \mapsto k-1)$; all other edges preserve it. The lifted graph must be acyclic. We write $\mathcal{C}(b, v_0)$ for this property.

4 Residualisation

The backlink rule appears only in the proof graph. A *residualisation* function $\mathcal{R}(b, v)$ reads back the graph into a CIC term by traversing the successors of v and replacing backlinks with binders:

- A vertex with no successors returns its own term.
- A vertex with one successor applies the driving step (unfold, β-reduce, apply constructor) to the successor's residual.
- A vertex that is the target of a backlink becomes a whose body is the residual of its successors, with recursive calls referencing the binder.

The result is a well-typed CIC term using only standard constructors (λ, application, , ,). No backlink constructs remain.

5 CIU equivalence

The soundness guarantee is *CIU equivalence* (contextual equivalence under all closing substitutions):

$$t \approx_{\text{ciu}} u \;\equiv\; \forall \sigma, v.\ (t[\sigma] \Downarrow v \iff u[\sigma] \Downarrow v)$$

Each local rule of the sequent calculus preserves CIU equivalence. The asynchronous rules are invertible: the conclusion is CIU-equivalent to its premise because each is a single reduction step (β, fix unfolding, ι). The split rule preserves CIU through the case analysis of a neutral scrutinee: substituting constructor patterns for the scrutinee in each branch yields a CIU-equivalent configuration. The back rule creates a fix binder at the target vertex.

Theorem 1 (CIU soundness). *If b is a rooted, ranked cyclic proof with root v and $\ell(b, v) = \Gamma \vdash t : A$, then*

$$t \approx_{\text{ciu}} \mathcal{R}(b, v).$$

Proof (Proof sketch). By well-founded induction on the budget trace. Each edge preserves CIU: asynchronous rules are single reduction steps, split preserves CIU through a neutral scrutinee, and backlinks become recursive self-references. The residualiser $\mathcal{R}$ unions these transformations across the graph. Full details are mechanised in Rocq (0 axioms) for the special case of graphs produced by the supercompiler [4]; the same proof structure generalises to any rooted, ranked cyclic proof because it depends only on the local rules and the trace condition, not on the supercompilation algorithm.

6 Admissibility

Theorem 2 (Backlink admissibility). *For every configuration graph b and vertex v,*

$$\mathcal{C}(b, v) \implies \exists\, t_{\mathsf{std}},\ t \approx_{\text{ciu}} t_{\mathsf{std}} \ \wedge\ t_{\mathsf{std}} = \mathcal{R}(b, v)$$

where $\ell(b, v) = \Gamma \vdash t : A$.

In words: every rooted cyclic proof can be read back to a standard CIC term (using only for recursion) that is observationally indistinguishable from the root goal.

Proof. Take $t_{\mathsf{std}} = \mathcal{R}(b, v)$. By construction (Section 4), t_{std} uses only standard CIC constructors—no backlinks remain. The CIU equivalence $t \approx_{\text{ciu}} t_{\mathsf{std}}$ is Theorem 1.

The mechanised proof (`BacklinkAdmissible.v`) derives admissibility as a four-line corollary of the CIU theorem for supercompiler-produced graphs; the proof sketch above covers the general case for arbitrary cyclic proofs.

7 Discussion

The admissibility transformation is the same residualisation that the proof graph already provides: traverse the graph, replacing backlinks with binders. The CIU theorem (proved for each local rule of the sequent calculus) establishes equivalence; the residualiser eliminates backlinks; the result follows immediately.

Compared to Brotherston and Simpson's result for first-order logic, our proof is simpler because (a) the graph is finite, and (b) residualisation is a syntactic transformation that explicitly replaces backlinks with , rather than requiring a semantic argument about proof unfoldings.

Acknowledgements. Thanks to James Brotherston for the question that motivated this paper.

References

1. Brotherston, J.: Cyclic proofs for first-order logic with inductive definitions. In: TABLEAUX (2006)
2. Brotherston, J., Simpson, A.: Sequent calculi for induction and infinite descent. In: LICS (2011)
3. Coquand, T., Huet, G.: The calculus of constructions. Information and Computation **76**(2–3), 95–120 (1988)
4. Mendel-Gleason, G.: Supercompilation corresponds to cyclic proof in the calculus of inductive constructions (2025), under review
5. Paulin-Mohring, C.: Inductive definitions in the system Coq: Rules and properties. In: Typed Lambda Calculi and Applications. pp. 328–345. Springer (1993)